LLD — Movie Booking Platform

Package Structure

```
com.xyz.moviebooking
 ├── model       → JPA entities
 ├── repository  → Spring Data interfaces
 ├── service     → Business logic
 ├── controller  → REST endpoints
 ├── dto         → Request / Response objects
 ├── discount    → Strategy pattern — pricing rules
 ├── exception   → Custom exceptions + global handler
 └── config      → Data seeder, Swagger
```

Domain Model

Movie

- id, title, language, genre, durationMinutes, rating, status

+ isActive() : boolean

Theatre

- id, name, city, active

- screens : List<Screen>

- amenities : Set<Amenity>

Screen

- id, name, screenType, totalSeats

- theatre : Theatre

- seats : List<Seat>

Seat

- id, rowLabel, seatNumber, seatType

- screen : Screen

+ getSeatCode() : String // "A3", "C12"

Show

- id, showDate, startTime, endTime, basePrice, status

- movie : Movie

- screen : Screen

+ isAfternoonShow() : boolean // startTime 12:00–16:59

ShowSeat

- id, price, status // AVAILABLE | LOCKED | BOOKED | BLOCKED

- show : Show

- seat : Seat

- booking : Booking // nullable — set on book, cleared on cancel

- lockedAt : LocalDateTime

[UNIQUE: show + seat]

Booking

- id, bookingReference, customerName, customerEmail, customerPhone

- numberOfTickets, baseAmount, discountAmount, finalAmount

- discountDescription, status, bookingTime

- show : Show

Discount Module

`<<interface>>` DiscountStrategy

+ isApplicable(ctx: DiscountContext) : boolean

+ calculate(ctx: DiscountContext) : DiscountResult

+ getStrategyName() : String

DiscountContext

- numberOfTickets, basePricePerTicket, totalBaseAmount
- show : Show

DiscountResult

- discountAmount, discountPercentage, description
~ noDiscount() : DiscountResult // static factory

AfternoonShowDiscountStrategy `implements DiscountStrategy`

+ isApplicable() → show.isAfternoonShow()
+ calculate() → 20% off totalBaseAmount
+ getStrategyName() → "AFTERNOON_SHOW"

ThirdTicketDiscountStrategy `implements DiscountStrategy`

+ isApplicable() → numberOfTickets >= 3
+ calculate() → 50% off (tickets / 3) qualifying tickets
+ getStrategyName() → "THIRD_TICKET"

DiscountEngine

- strategies : List<DiscountStrategy> // all @Component impls injected by Spring
+ getBestDiscount(ctx) : DiscountResult // evaluates all, returns highest discount

Service Layer

BookingService

+ bookTickets(request) : BookingResponse @Transactional

validate show → lock seats (SELECT FOR UPDATE)

→ sum prices → apply best discount → persist

+ cancelBooking(reference) : void @Transactional

cancel booking → reset ShowSeat to AVAILABLE

+ getBookingByReference(ref) : BookingResponse

+ getBookingsByEmail(email) : List<BookingResponse>

ShowService

+ createShow(request) : Show @Transactional

validate → conflict check → persist

→ bulk-insert ShowSeat for every Seat in Screen

+ updateShow(id, request) : Show @Transactional

You can only edit a show if nobody has booked yet. If you change the time, we'll verify it doesn't clash with another show. If you change the price, we'll update unsold seats only.

+ cancelShow(id) : void @Transactional

Cancel the show. Any unsold seats are closed out. Sold seats are handled through the refund/cancellation process separately.

+ getAvailableSeats(showId) : List<SeatResponse>

- calculatePrice(base, type) : BigDecimal // REGULAR 1× | PREMIUM 1.5× | VIP 2× | RECLINER 2.5×

Relationships

Theatre 1 —— N Screen (owns)

Screen 1 —— N Seat (owns — permanent layout)

Movie 1 —— N Show

Screen 1 —— N Show

Show 1 —— N ShowSeat (created on show creation, one per Seat)

Seat 1 —— N ShowSeat

Booking 1 —— N ShowSeat (set on booking, cleared on cancel)

Theatre N —— N Amenity (join table)

BookingService —▶ DiscountEngine —▶ List<DiscountStrategy>

Design Patterns

Strategy — Discount Rules

Each discount rule is a separate `@Component` behind `DiscountStrategy`.

`DiscountEngine` collects all implementations at startup — adding a new rule = one new class, nothing else changes.

Repository — Data Access

Services depend on repository interfaces, never on JPA/SQL directly.

Keeps business logic decoupled from persistence; repositories are trivially mockable.

DTO — API Boundary

Entities never cross controller boundaries.

`BookingRequest / BookingResponse` define the contract, entity structure is free to change internally.

Layered Architecture

`Controller → Service → Repository` — each layer has one responsibility.

A new delivery channel (gRPC, messaging) only touches the controller layer.

Factory Method — Seat Pricing

`calculatePrice(base, SeatType)` centralises all multiplier logic in one switch expression inside `ShowService`.

Design Principles

Single Responsibility

`BookingService` owns booking transactions. `ShowService` owns show lifecycle. `DiscountEngine` owns discount selection. Each class has exactly one reason to change.

Open / Closed

`DiscountEngine` is closed for modification. New discount rules extend it by adding a new class — zero changes to existing code.

Dependency Inversion

Services depend on repository *interfaces* and `DiscountEngine` depends on `List<DiscountStrategy>` — never on concrete classes.

DRY — Seat price logic

Price multiplier logic lives in exactly one place: `ShowService.calculatePrice()`. Both `createShow` and `updateShow` call it — never duplicated.